

Matkamuistot

Amaru ostaa matkamuistoja vieraasta kaupasta. Matkamuistoja on N eri **tyyppiä**. Kaupassa on saatavilla jokaista tyyppiä loputon määrä.

Jokaisella matkamuistotyyppillä on tietty hinta. Tyypin i ($0 \leq i < N$) matkamuiston hinta on $P[i]$ kolikkoa, jossa $P[i]$ on positiivinen kokonaisluku.

Amaru tietää, että matkamuistotyyppit ovat hinnan mukaan laskevassa järjestyksessä ja että kaikki hinnat eroavat toisistaan. Tarkemmin sanottuna $P[0] > P[1] > \dots > P[N-1] > 0$. Lisäksi hän on saanut tietää arvon $P[0]$. Valitettavasti Amarulla ei ole mitään muuta tietoa matkamuistojen hinnoista.

Ostaakseen matkamuistoja Amaru tekee useita kauppvoja myyjän kanssa.

Kukin kauppa koostuu seuraavista vaiheista:

1. Amaru antaa myyjälle jonkin (positiivisen) määrän kolikoita.
2. Myyjä laittaa kolikot kasaan takahuoneen pöydälle, josta Amaru ei näe niitä.
3. Myyjä tarkastelee kutakin matkamuistotyyppiä $0, 1, \dots, N-1$ tässä järjestyksessä, yksi kerrallaan. Jokaista tyyppiä tarkastellaan **tasan kerran** kaupan aikana.
 - Tarkastellessaan matkamuistotyyppiä i , jos kolikoita on kasassa vähintään $P[i]$, niin
 - myyjä poistaa kasasta $P[i]$ kolikkoa ja
 - myyjä asettaa pöydälle yhden tyypin i matkamuiston.
4. Myyjä antaa Amarulle kaikki kasassa jäljellä olevat kolikot ja pöydällä olevat matkamuistot.

Huomaa, että pöydällä ei ole kolikoita tai matkamuistoja ennen kaupan alkua.

Tehtävänäsi on ohjeistaa Amarua suorittamaan jokin määrä kauppvoja siten, että:

- jokaisessa kaupassa hän ostaa **vähintään yhden** matkamuiston ja
 - yhteensä hän ostaa **tasan** i kappaletta tyypin i matkamuistoja jokaiselle i , jolle $0 \leq i < N$.
- Huomaa, että tämä tarkoittaa, että Amaru ei saa ostaa yhtäkään tyypin 0 matkamuistoa.

Amarun ei tarvitse minimoida kauppvojen määrää, ja hänellä on loputtomasti kolikoita.

Implementaatioyksityiskohdat

Sinun tulee implementoida seuraava aliohjelma.

```
void buy_souvenirs(int N, long long P0)
```

- N : matkamuistotyyppien määrä.
- P_0 : matkamuistotyyppin 0 hinta $P[0]$.
- Tätä aliohjelmaa kutsutaan tasan kerran jokaisessa testissä.

Yllä oleva aliohjelma voi kutsua seuraavaa aliohjelmaa ohjeistaakseen Amarua tekemään kauppvoja:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : Amarun myyjälle antamien kolikkojen määrä.
- Aliohjelma palauttaa parin. Parin ensimmäinen alkio on lista L ostetuista matkamuistotyypeistä (kasvavassa järjestyksessä). Toinen alkio on kokonaisluku R , kaupan jälkeen Amarulle palautettujen kolikkojen määrä.
- Vaaditaan, että $P[0] > M \geq P[N-1]$. Ehto $P[0] > M$ takaa, että Amaru ei osta tyyppin 0 matkamuistoja, ja ehto $M \geq P[N-1]$ takaa, että Amaru ostaa vähintään yhden matkamuiston. Jos nämä ehdot eivät täyty, ratkaisusi saa tuomion `Output isn't correct: Invalid argument`. Huomaa, että erona arvoon $P[0]$ arvoa $P[N-1]$ ei anneta syötteessä.
- Aliohjelmaa kutsutaan enintään 5000 kertaa jokaisessa testissä.

Arvostelija ei ole adaptiivinen. Siten lista matkamuistojen hinnoista P on kiinnitetty ennen kuin aliohjelmaa `buy_souvenirs` kutsutaan.

Rajat

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ jokaiselle i , jolle $0 \leq i < N$.
- $P[i] > P[i+1]$ jokaiselle i , jolle $0 \leq i < N-1$.

Osatehtävät

Osatehtävä	Pisteet	Lisäehdot
1	4	$N = 2$
2	3	$P[i] = N - i$ jokaiselle i , jolle $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ jokaiselle i , jolle $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ jokaiselle i , jolle $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ jokaiselle i , jolle $0 \leq i < N - 1$.
6	33	Ei lisäehtoja.

Esimerkki

Tarkastellaan seuraavaa kutsua.

```
buy_souvenirs(3, 4)
```

On olemassa $N = 3$ matkamuistotyyppiä, ja pätee $P[0] = 4$. Huomaa, että on vain kolme vaihtoehtoa listalle P : $[4, 3, 2]$, $[4, 3, 1]$ ja $[4, 2, 1]$.

Oletetaan, että aliohjelma `buy_souvenirs` tekee kutsun `transaction(2)`. Oletetaan lisäksi, että kutsu palauttaa $([2], 1)$, mikä tarkoittaa, että Amaru ostaa yhden tyyppin 2 matkamuiston ja myyjä antaa hänelle takaisin yhden kolikon. Huomaa, että tästä voidaan päätellä, että $P = [4, 3, 1]$, sillä:

- hinnoille $P = [4, 3, 2]$ kutsu `transaction(2)` olisi palauttanut $([2], 0)$,
- hinnoille $P = [4, 2, 1]$ kutsu `transaction(2)` olisi palauttanut $([1], 0)$.

Tämän jälkeen aliohjelma `buy_souvenirs` voi tehdä kutsun `transaction(3)`, joka palauttaa $([1], 0)$, mikä tarkoittaa, että Amaru ostaa yhden matkamuiston tyyppiä 1 ja myyjä antaa hänelle takaisin 0 kolikkoo. Tähän mennessä hän on saanut yhteensä yhden matkamuiston tyyppiä 1 ja yhden matkamuiston tyyppiä 2.

Lopuksi aliohjelma `buy_souvenirs` voi tehdä kutsun `transaction(1)`, joka palauttaa $([2], 0)$, mikä tarkoittaa, että Amaru ostaa yhden matkamuiston tyyppiä 2. Huomaa, että tässä oltaisiin myös voitu tehdä kutsu `transaction(2)`. Tähän mennessä hän on saanut yhden matkamuiston tyyppiä 1 ja kaksi matkamuistoa tyyppiä 2, kuten vaadittiin.

Malliarvostelija

Syöteen muoto:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Tulosteen muoto:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Yllä $Q[i]$ vastaa tyyppiä i ostettujen matkamuistojen yhteismäärää jokaiselle i , jolle $0 \leq i < N$.